New Course → FASTAPI (9 hours) → lagatar till then no priority

Next → saftey & Guardrails:

ML API

FastAPI industry grade

Now FastMCP

FastMCP For ML

Part-1

Fundamentals of FastAPI using project (1-7) videos.

Part-2

FastAPI + ML

Part-3

Deployment

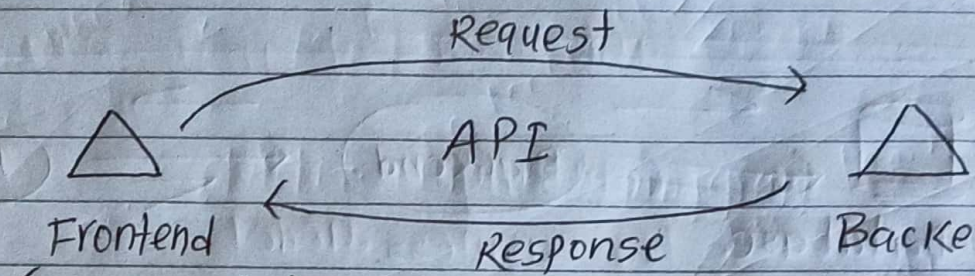
↓

Docker + aws.

you can then make GenAI / Agentic AI Model & Can Deploy it (aws + Docker) ↳ industrygrade

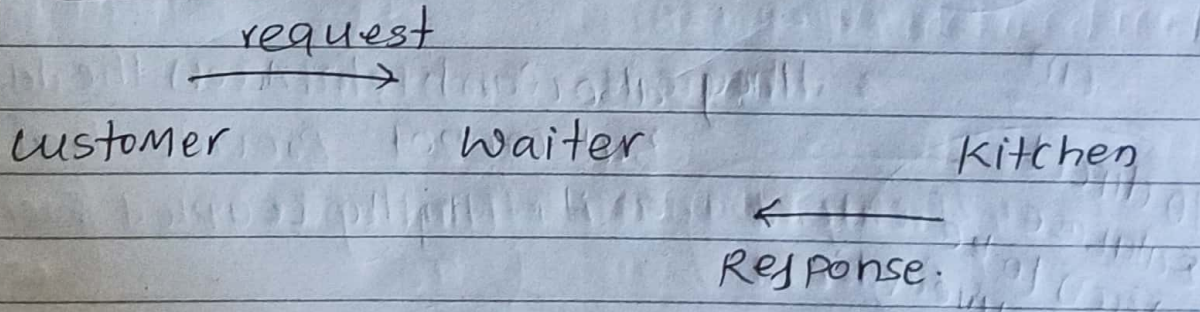
Scanned with OKEN Scanner

What is an API?



{ A connector between two pieces of software }

Analogy Resturant

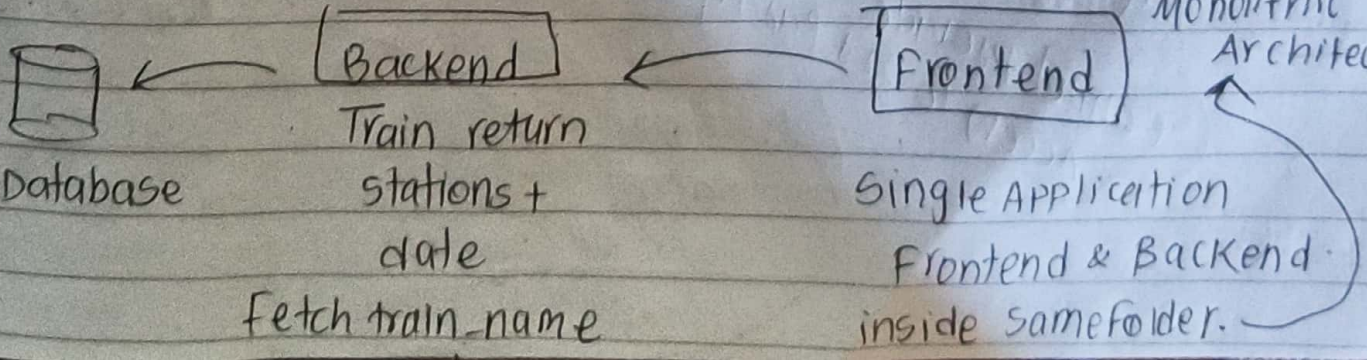


Why API exists: (Need For APIs)

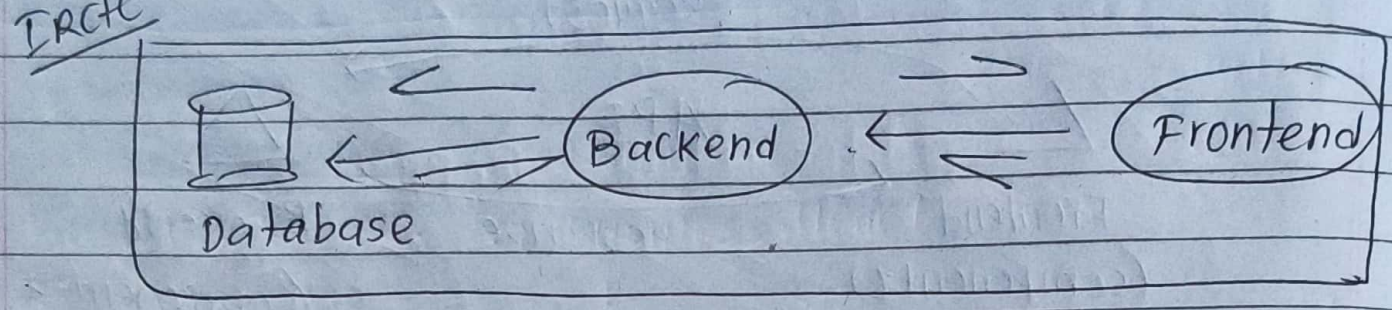
Era Before API's:

Pre API Era

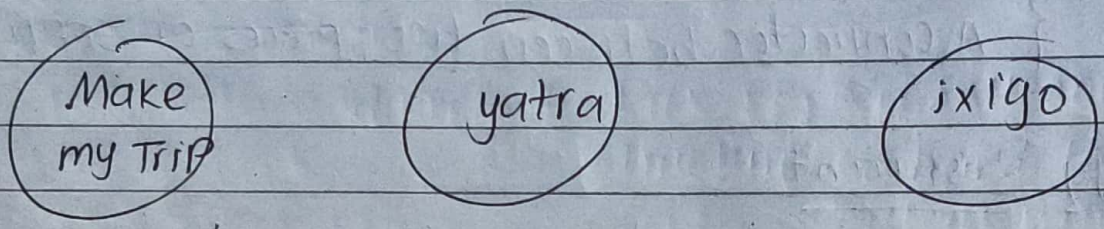
IRctc Just display train information:



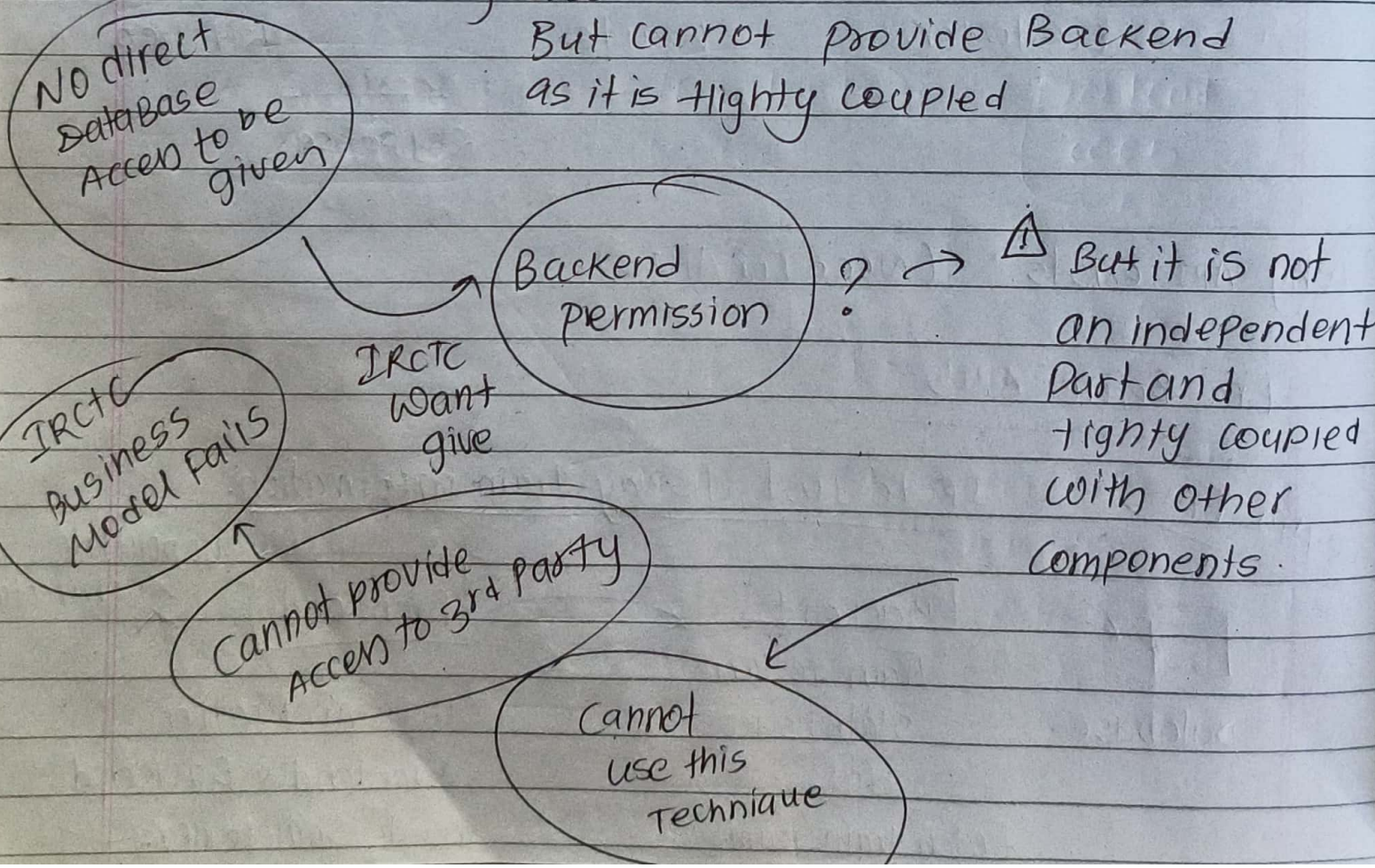
Problem with Monolithic Architecture:



Monolithic structure.

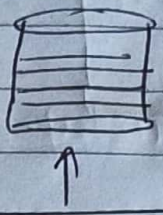


they also want to Accen the database:
But cannot provide Backend as it is Highly coupled



Scanned with OKEN Scanner

Solution decouple Backend



Backend

Fetch-trains Func().

Some special Functions exposed in internet

API

/trains

internet

Call this

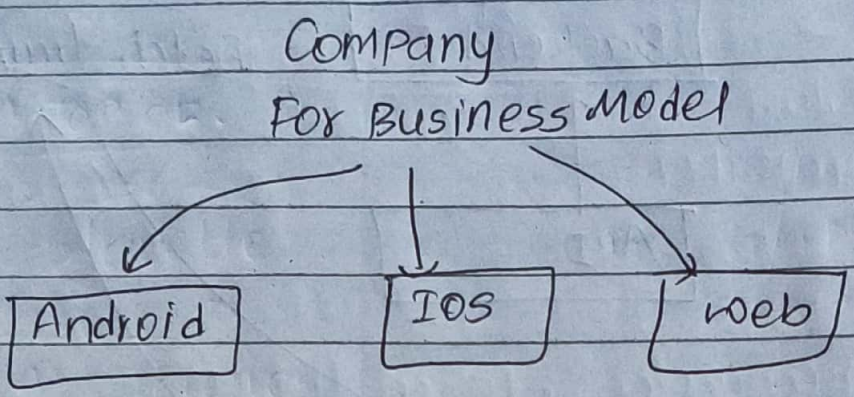
{ Backend is Publicly available due to endpoints named API.

Can Apply constraints. }

Frontend is also now 3rd Party & should have to use API to communicate with API

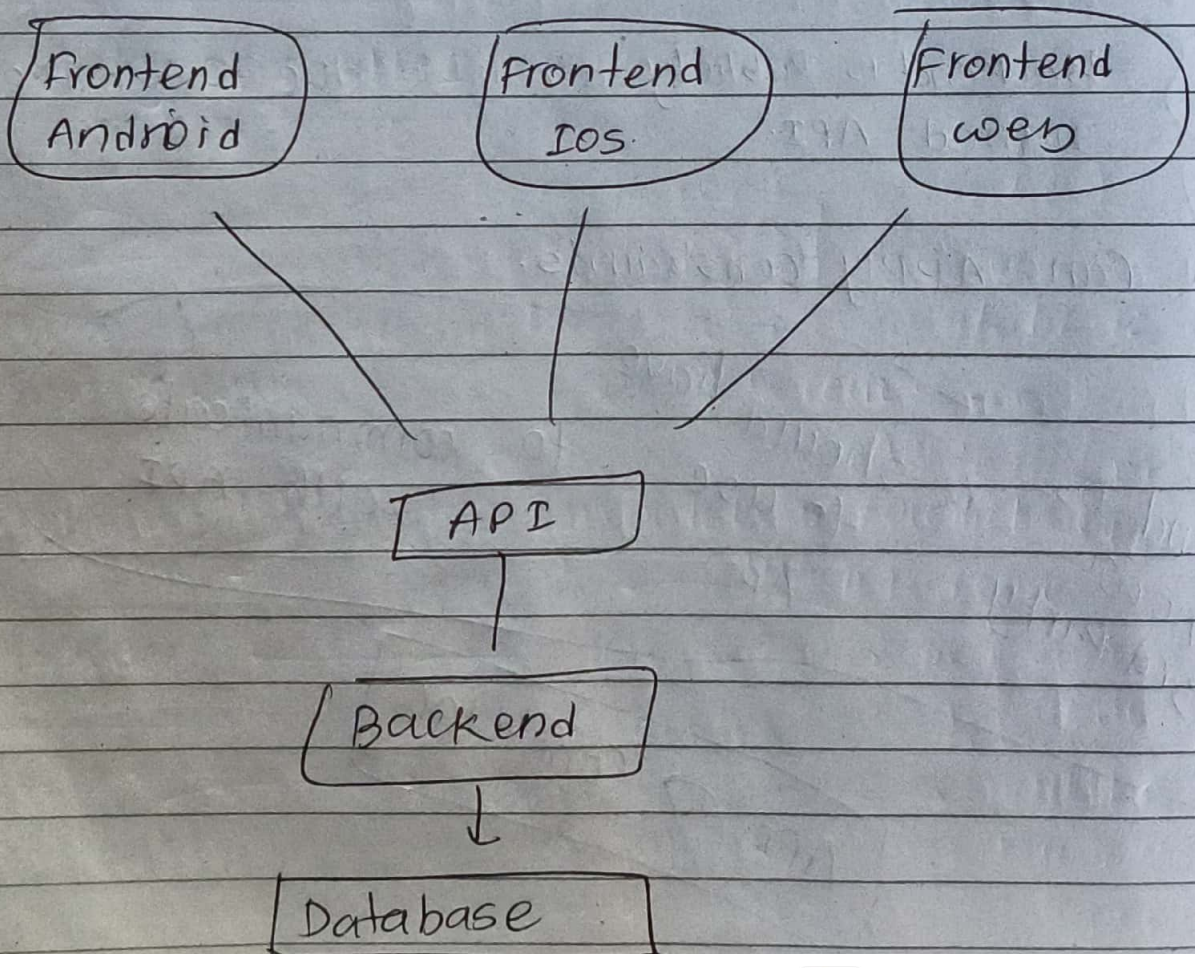
API → Connector

Protocol → Request → HTTP request
Data format → response → json.



different stack for the Monolithic structure. So hard to sync.

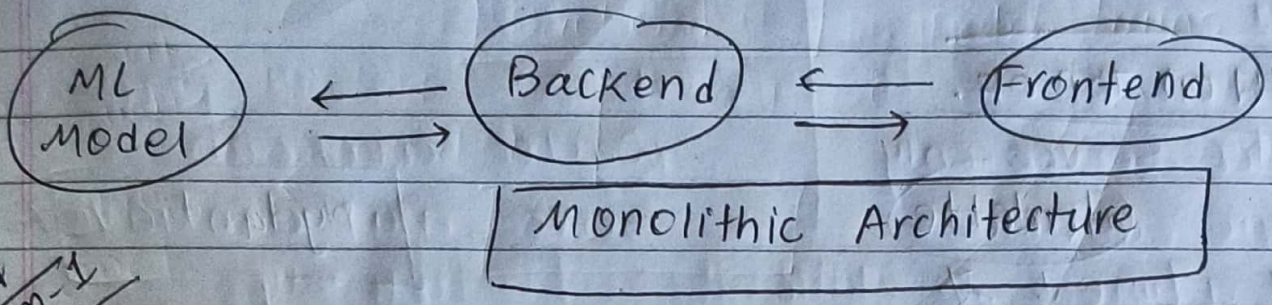
Soln 3 different Frontend



Scanned with OKEN Scanner

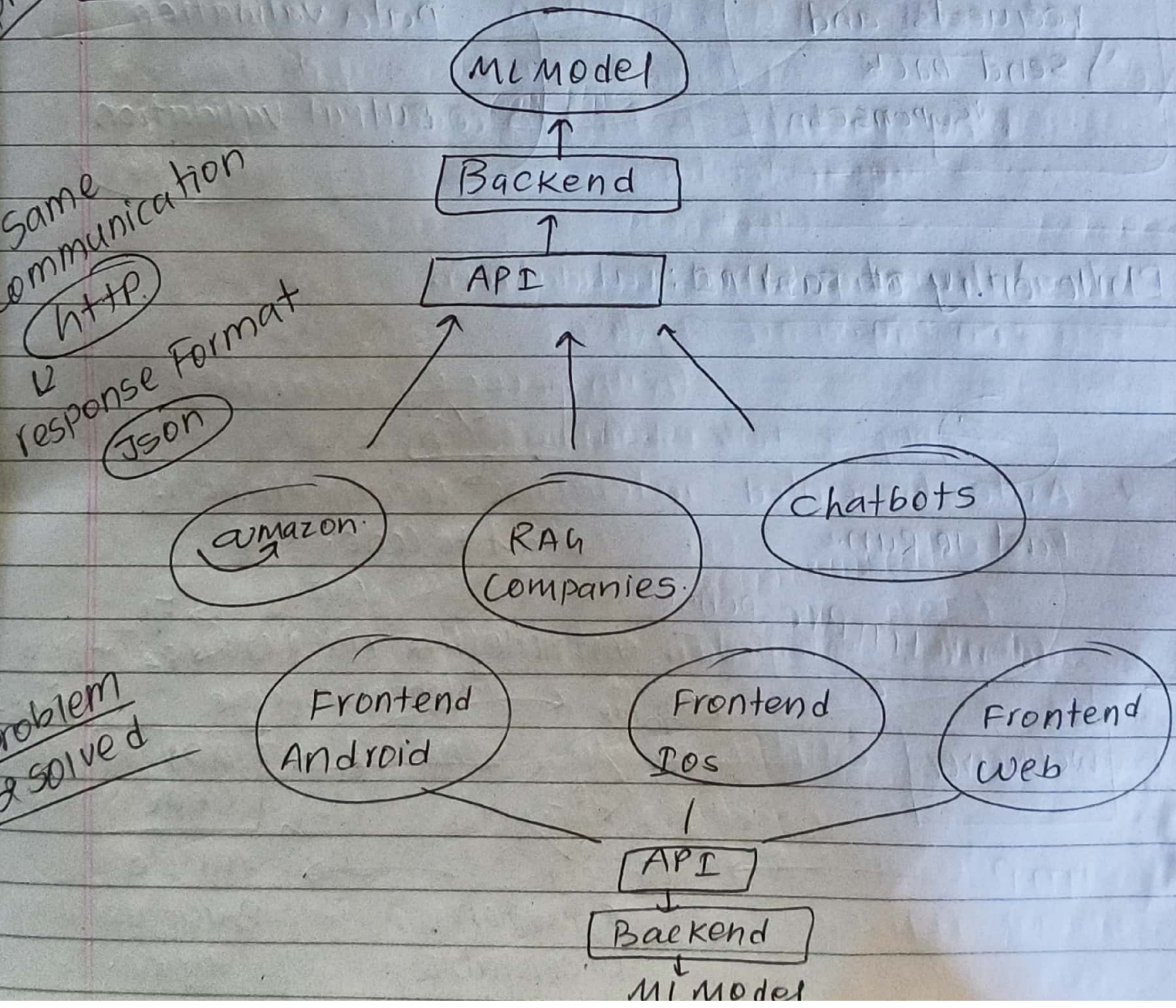
AI-ML perspective

V.IMP → ML Model.



Solved Problem-1

Same Problem For Business Scaling:

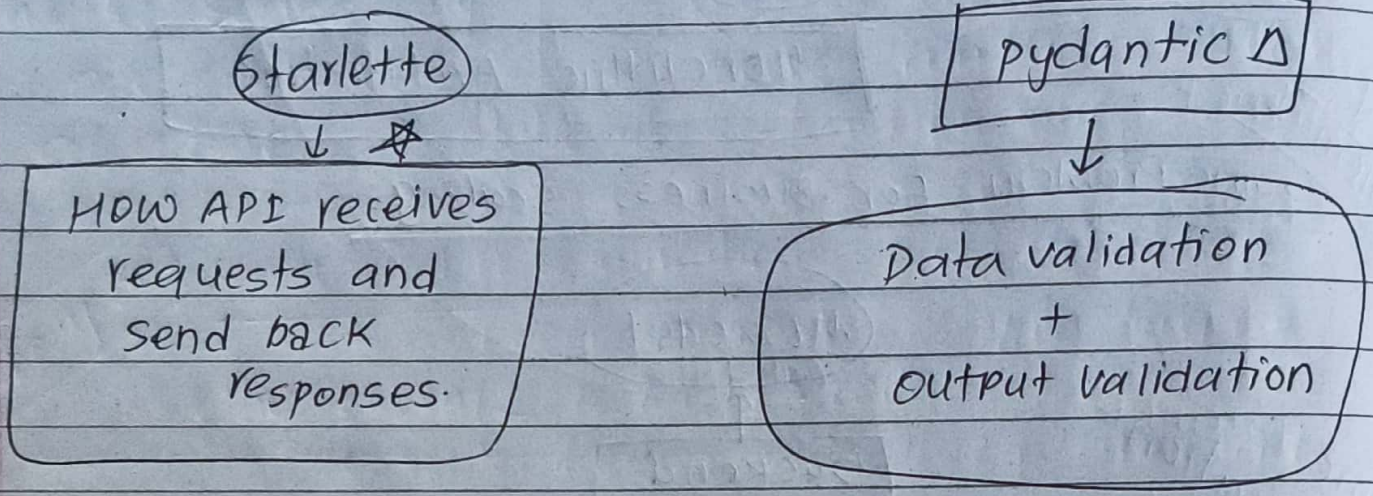


V-IMP to cover

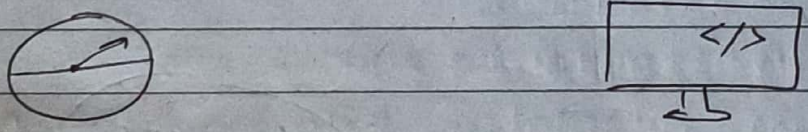
FASTAPI Philosophy

↳ Buid Industry grade API with Python.

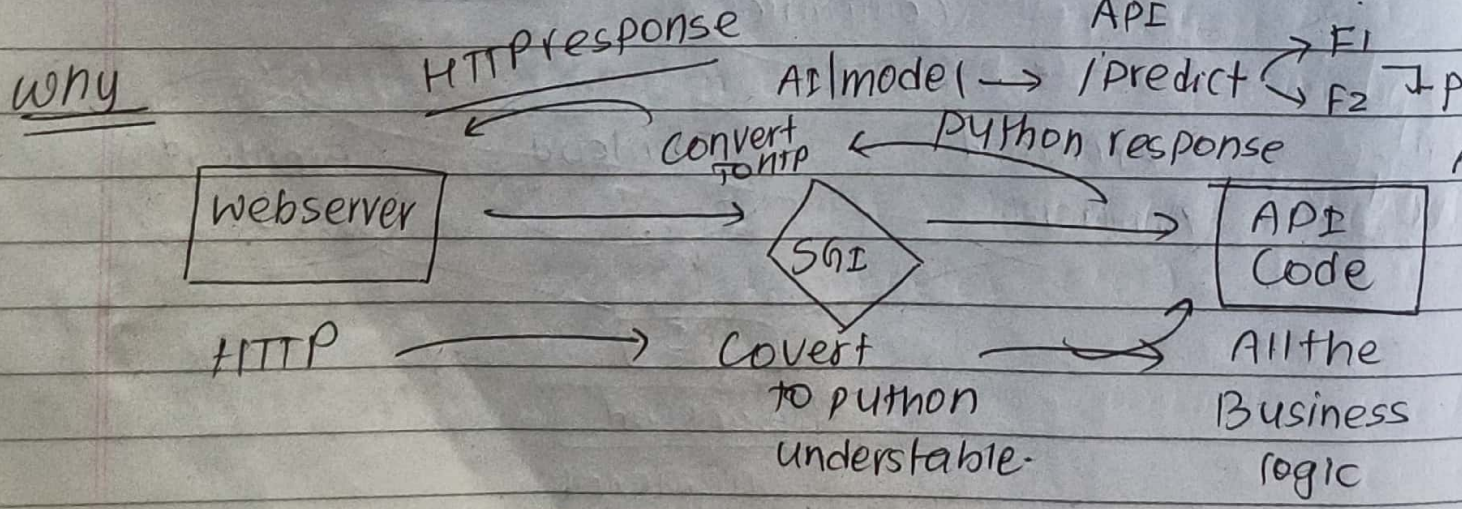
Build on TOP OF.



Philosophy OF FastAPI:



why

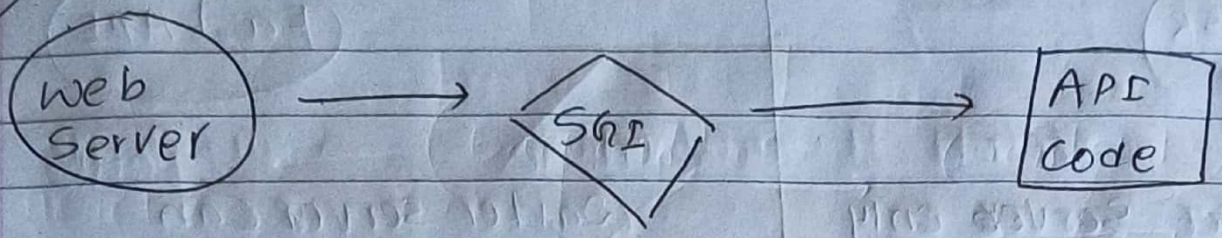


Scanned with OKEN Scanner

Philosophy no: 1

How previous library (~~Fast~~ Flask used it)

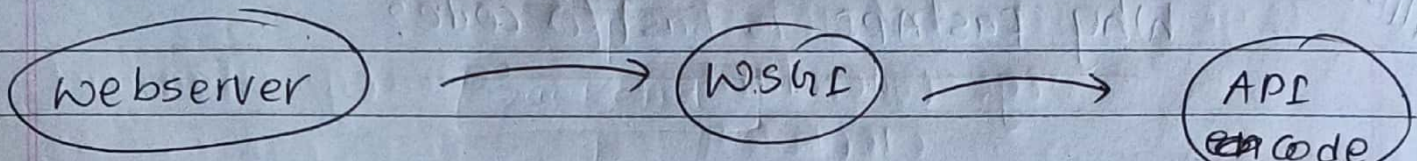
Flask



Service Gateway interface:

SGI Types → WSGI (synchronous nature)

→ ASGI (Asynchronous nature).

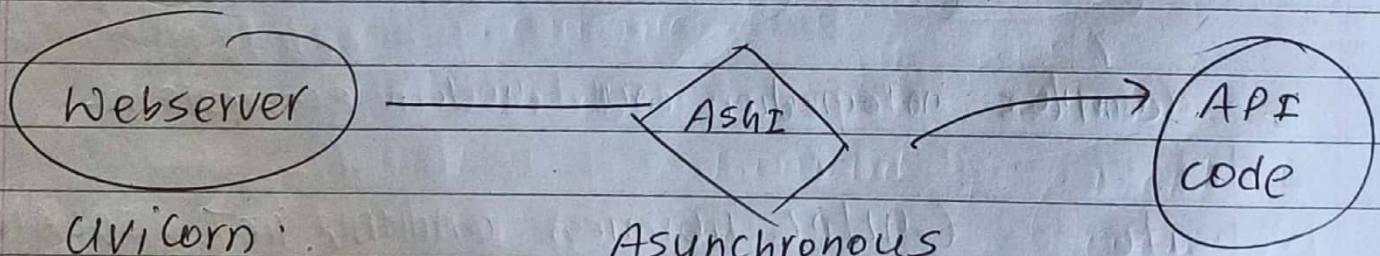


[Gunicorn] Also synchronous

Werkzeug synchronous

sync. code.

Fast



Uvicorn. (Asynchronous).

Asynchronous ↓ Starlette. Asynchronous.

Async await Feature implemented.

Analogy between Fast API & Flask API

Flask API	Fast API
Waiter Server only one customer at a time	Waiter server can Serve multiple server.

philoshopy no2:

Why FastAPI is Fast to code?

1. Automatic input validation.
   ⇓
   (with help of pydantic)

2. Auto-Generated interactive documentation:

3. Seamless integration with modern ecosystem.

   (ML/DL libraries, OAuth, JWT, SQLAlchemy, Docker, Kubernetes etc.)

Scanned with OKEN Scanner

Setup:

```
python -m venv myenv
myenv\Scripts\activate
```

```
pip install uvicorn fastapi pydantic
```

main.py

run → uvicorn main:app --reload

IMP

HTTP Methods in FastAPI

Project overview:

Patient1 info

Patient2 info

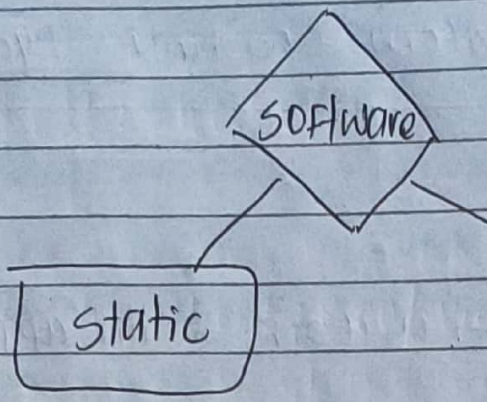
endpoints

```
/create
/view
/view/patient_id
/update/patient_id
/delete/patient_id
```

Scanned with OKEN Scanner

HTTP Methods:

Based upon user interaction:



Minimal communication
one way communication

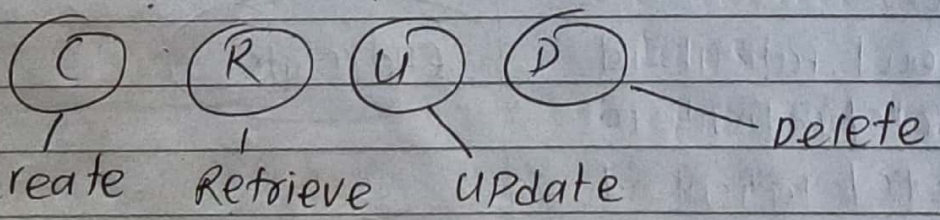
eg: Calendar.
clock

Maximum comm.
two way "

eg: MS word.

Focus on dynamic software

only 4 type of interaction:

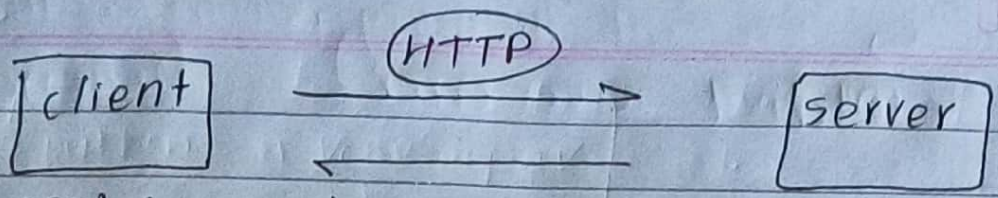


Website

↳ Are also software

Special software installed on other machine & used in other machine.

Normal soft → same machine install & use.



software used

software installed

Website

Static Website

dynamic website

eg: Blog & Goverment info Website

eg: instagram

But only 4 operations (interaction) are done

CRUD

→ HTTP → verb → Get (~~Retrieve~~)
→ HTTP → verb → POST (~~Create~~)
→ HTTP → verb → Put
→ HTTP → verb → Delete.

Scanned with OKEN Scanner

V. Imp

Path & Query Params:

Path Params:

localhost:800/view ⟶ all Patients.

⟶ localhost:800/view/3 ⟶ Dynamic specific resource locate.
4

if you want to (reterive, update, delete) specific resource use path params.

entrance user exp 1

path() Function

used to provide Metadata, validation rules & documentation hints.

```
def view_patient (patient_id: str = Path(..., desc ))
```

↑ Means path parameter is required.

Important concept

HTTP status code:

↳ 3-digit numbers.

Know these 4 types only

What happend to the http request:

- 2xx → Success → request was successfully retrieved & processed.
- 3xx → redirection → Further action needs to be taken.
- 4xx → ⚠ client error
- 5xx → x server Error

200 OK	standard success	A Get or Post ✓
201 OK	Resource created	Post created ✓
204 No content	success but no data returned	After a delete request.
400 Bad request	invalid req.	
401 unauthorized.	Nol invalid authentication.	
403 Forbidden.	Authenticated but no permission.	
404 Not found	Resource doesn't exist.	

Scanned with OKEN Scanner

(500 internal server error) Generic Faliure

502 Bad gateway

503 Service unavaliable.

problem till Now

try with patient not in the Json.

Code	Response Body
200	{ 'error': 'patient not Found' }

to solve this

We need custom exception class

HTTPException

HTTPException:

special Builtin exception in FastAPI used to

return custom HTTPerror responses

When something goes wrong in your API.
Instead of returning a normal JSON or crashing the server you can gracefully raise an error with.
a proper Code, custom error message, extra headers.

Scanned with OKEN Scanner

Next v.imp

Query params

/view → All Patients data. Chronologically

give power of sorted manner.

ascendent & decending order.

sortby & order → Asc|DESC optional values.

/patients (?) city = Delhi & sort_by = age

After question mark — Key value pairs seperated by &

Query parameters are optional key-value pair appended to the end of a URL used to pass additional data to the server in an HTTP request.

---

end point for sorting patients.

---

with the help of query parameter

→ query

- sortby
  - weight, height, Bmi
- order
  - ASC
  - DESC

Scanned with OKEN Scanner

Query() → utility function — Similar to path

↳ declare
↳ validate
↳ document the query parameters in your API endpoints

~~@app.get('/sort'):~~

✓ From fastapi import FastAPI, Query, HTTPException, Path

@app.get('/sort')
def sort_patients(sort_by: str = Query(..., description =
'sort on the basis of height, weight or bmi'),
order: str = Query ('asc', description = 'sort
in asc or desc order')):

valid_Fields = ['height', 'weight', 'bmi']

Import
Try
✓ implement

if sort by not in valid_fields:
raise HTTPException (status_code = 400,
detail = F'invalid field select from
{valid_fields}')

query params in

if order not in ['asc', 'desc']:
raise _____

sort_order = True if order == 'desc' else 'false'
data = load_data()

Sorted_data = Sorted (data.values(), ~~key~~
~~key = lambda x.get(~~
key = lambda x: x.get (sort_by, 0)
, reverse = ~~False~~ sort_order)

return sorted_data.

Scanned with OKEN Scanner

Path Params	V.S	Query params.
→ url's dynamic part you can fetch particular resource help in retrieval update deletion & Always required ,= Path(...,		⇒ existing endpoints. ⇓ searching Filtering pagination ⇒ params are optional

you can use ~~All~~ too of them in a single endpoint

Next Part
we use it together

learn in depth.

Next Big thing
Pydantic Data validation

V-IMP

Pydantic → start From First principle learning Method.

Why Pydantic:

problem:

Senior programmer Code.

```
def insert_patient_data(name, age):

    print(name)
    print(age)
    print('inserted into database')
```

Junior programmer:

```
insert_patient_data('nitish', 'thirty')
```

Works Fine.

But Senior expect → 30.

one solution

- Type hinting

```
def insert_patient_data(name:str, age:int):
    print(name)
    print(age)
    print('inserted into database')
```

still

```
insert_patient_data('nitish', 'thirty')
```

✓

Works Fine.

Scanned with OKEN Scanner

2nd solution
own Typed validation

Senior programmer

```
def insert_patient_data(name:str, age:int):
    IF type(name)==str and type(age)==int:
        print(name)
        print(age)
        print('inserted into database')
    else:
        raise TypeError('incorrect data type')
```

Junior programmer

```
insert_patient_data('nitish', 'thirty')
```

↳ raise TypeError

Works Fine But NOT Scalable

Python donot have Typed Validation By default

Problem 2:

Data validation

what if age is -ve.

☆ pydantic help with [Data validation. + Typed validation.]

Scanned with OKEN Scanner

How pydantic solve 2 problem:

Step1: Define Pydantic schema.

Step2: create object & pass raw value

Automatically do Data validation + Typed validation.

Step3: pass this validated Pydantic data into the Function.

⇒ Pip install Pydantic → version 2

step1 From Pydantic import BaseModel

```
Class Patient (BaseModel):
    name: str
    age : ~~str~~ int
```

this is scalable & smart Also provide Typed validation.

smart works For 'age': '30'

step2

```
PatientInfo = {'name': 'nitish', 'age': 30}
patient1 = Patient (** patientInFo)
```

→ unpack.

Pydantic object

step3

```
def insert_patient_data (patient: Patient):
    print (patient.name)
    print (patient.age)
    print ('Inserted')
```

Scanned with OKEN Scanner

Important concept:

From typing import List, Dict

#Donot use list & dict ? Why.

```
allergies: List[str]
contact_details: Dict[str, str]
```

Because two level validation.

List[str] ✓ ✓ list[str] ✓ ✗

Required vs Optional:

By default all of fields are required.

From typing import Optional

allergies: Optional[list[str]] = None

↑ should provide default value.

V.Imp Data validation

(1) Pydantic Builtin Datatypes.

From pydantic import Emailstr.

email: Emailstr. → Built in custom Data type

Similarly AnyUrl, custom data type.

Scanned with OKEN Scanner

2nd Method

According to your Business usecase: custom data validation.

Field

```
From Pydantic import Field

weight: Float = Field( gt=0 , lt=120 )

name: Float = Field( max_length= 20 )

allergies: optional[List[str]] = Field(max_length=5)
```

Not only for data validation

↓ you can Attach Metadata.

using Annotated + Field.

```
From typing import Annotated.

name: Annotated [str, Field(max_length=50,
                            title = 'Name of the patient',
                            description '. Give the name of
                                          patient',
                            examples = ['raju',])

married: Annotated[bool, Field ( default=None,
                                 description = 'weight')
                  )
```

Scanned with OKEN Scanner

Pydantic & Type Coearsion

Weight : Float ✓ Not Always good

`'Weight': '75.2'` ✓ Type coearsion

to suppress Type Coearsion: suppress

```
Weight: Annotated[Float, Field(gt=0, strict=True)]
```

Field validator

Validate patient from hdfc & give 20% Dis.

1st use case

```
From pydantic import field_validator

class patient(BaseModel):

    Email: EmailStr

    @field_validator('Email')
    @classmethod
    def email_validator(cls, value):
        valid_domains = ['hdfc.com', 'icici.com']
        # abc@gmail.com
        domain_name = value.split('@')[-1]

        if domain_name not in valid_domains:
            raise ValueError('Not a valid Number')

        return value
```

2nd usecase:

```
@Field_validator('name')
@ classmethod
def transform_name(cls, value):
        return value.upper()
```

1st use case:

→ using field validator Complex data validation can be done.

2nd usecase:

→ using field_validator you can transform fields.

★ Field-validator operating mode

All validation + Type coercion.

Patient = Patient(** Patient_details)

Before Mode	After Mode
@ Field_validator('name', mode='before')	By default: after @ Field_validator('name', mode)
@ classmethod	def transform(cls, value)
def transform(cls, ~~name~~ value)	↑ this value you will get after Type coercion.
↑ this value you will get Before Type coersion.	

Scanned with OKEN Scanner

V. Imp

One problem arise

Flaw Big

What if data validation depends on more than 1 Fields

* implement validation if patient age is > 60 then in his contact_details atleast one contact number should be there.

failed. Field_validator only validate only one Field. can

solution

Model validator

combine (multiple Fields) & perform data validation.

step 1:

```
From Pydantic import model_validator
Class patient (BaseModel):
    age : int
    Contact_details : ~~Optional~~ Dict [str, str]
                      ~~Annotated[Optional~~
```

step 2

```
@model_validator (mode='after')        # no args. Because Validation on entire pydantic model.
def validate_emergency_contact ( cls, model)        # model ↑ pydantic model
    if model.age > 60 &
        'emergency' not in model.contact_details :
        raise ValueError ('Patients older than 6
    return model.
```

V. IMP

Computed Field ?

```
From pydantic import computed_field.
class Patient (BaseModel):

    weight: float
    height : float

    @computed_field
    @property
    def calculate_bmi (self) -> float:
        Bmi = round (self.weight / self.height **
        return Bmi
```

Nested Model

Using one ~~Field~~ model inside another as a Field.

```
class Patient (BaseModel):
    address: Address

class Address (BaseMode
    city: str
    state: str
    pincode : str
```

~~address_dict = {'city': 'guru~~

- Better organization
- Reusability
- Readability
- validation

Using dot operator

Can Find

Serialization v-IMP concept

patient1.model_dump()

~~python~~ Pydantic object to python dictionary.

patient1.model_dump_json()

Pydantic object to Json.

patient1.model_dump(include=['name']) ✓ only name

patient1.model_dump(exclude=['age']) All except age.

exclude_unset = True → Don't include Pydantic default values.

Now practice All of above

V-Imp

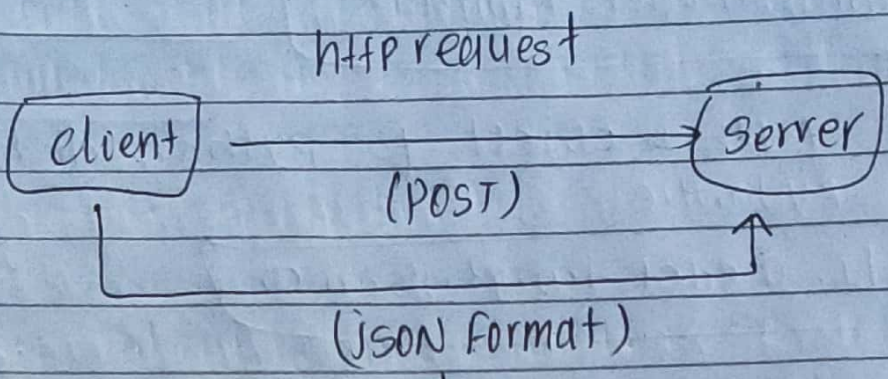
HTTP post

Crud

↳ create endpoint

json.load(F)

Step1:



Step2

Validate using Pydantic Model → if not → error raise

↓ yes (valid)

Step3

json File → New record add:

Write the Code & Create .ipynb File

Read

```
with open ('patients.json', 'r') as F:
    data = json.load (F)
return data
```

write

```
with open ('patients.json', 'w') as F:
    json.dump (data, F)
```

Scanned with OKEN Scanner

put & delete

CRUD → delete (delete)
↓
update (put)

put & delete

update endpoint:

'/edit' → HTTP → PUT

Patient_id

Request Body

create a new pydantic schema

Fields with optional feature

② New_data → existing data → change

class Patient_update (

end point

Pydantic obj

```
@app.put('/edit/{patient_id}')
def update_patient(patiend_id: str, patient_update: PU)

    data = load_data()

    IF patient_id not in data:
        raise HTTPException(404, detail='Patient Not Found
    existing_patient_info:=
    data[pateint_id]
    Udated_patient_info=
    Patient_update.model_dump(exclude_unset=True)

    For key,value in udated_patient_info.items():
        existing_patient[key]= ~~updated~~ value.

// data[patient_id] = existing_patient_info.
// But Bmi & verdict still unchanged
```

So First create Pydantic object & then convert into da

existy Patent_into[id] = Patient_id.

patient_pydantic_obj. = ~~Pat~~ patient(**existy patient)

Patient_pydantic obj. model_dump(exclude(id)

data[key] = e

Swe (data)

Scanned with OKEN Scanner

delete patient_id as path params

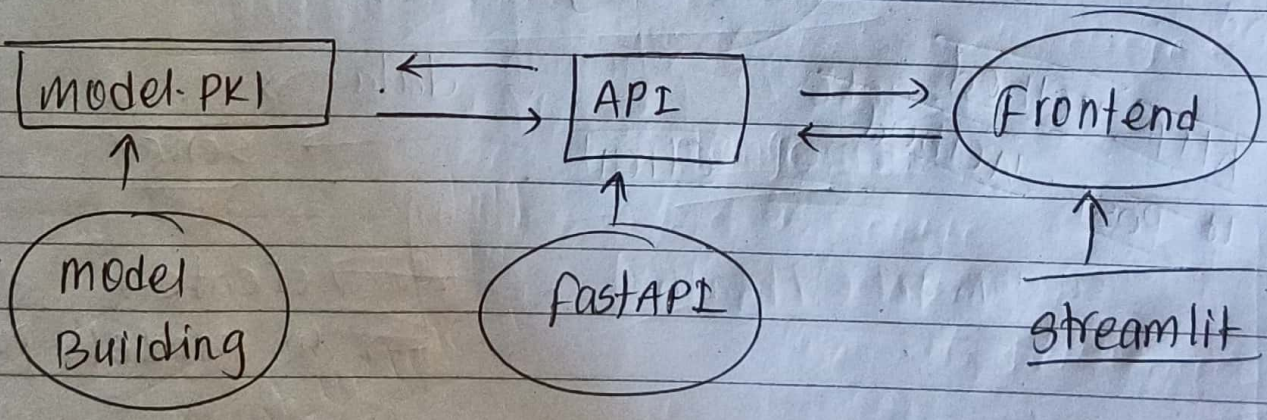
```
@app.delete('/delete/{patient_id}')
def delete_
```

FastAPI fundamentals done

FastAPI + ML

Part 1 — ML Model → Part 2 — fastapi → Part 3 — Frontend.

Flow diagram



Scanned with OKEN Scanner

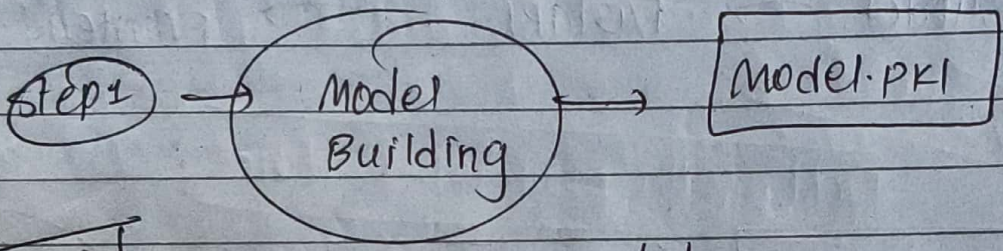
Step 1

Building & Exporting the Model:

Useful For { insurance company + Invididuals. }

Feature engineering

Toydataset



http method

to Post method

Client → data → server

→ create
→ ML model predict.

/Predict POST Request Body

{ ASK raw data: & Transform them.

create endpoint

```
# import model
import pickle
with open ('model.pkl', 'rb') as f:
    model = Pickel.load(f)

app = fastAPI()

# pydantic Model to validate the user input:

class userInput (BaseModel):

    // create Basic pydantic schema:
    // write compute_field to calculate Bmi,..

@app.post('/Predict')
def predict_premium (data: userInput):

    Pass data to the Model as Data Frame

input_df = pd.DataFrame([ {
        'bmi': data.bmi

    }])

predict  model.predict(input_df)[0]  → Return JsonResponse (statuscode: 200, content= )
```

improve API → production grade → Docker
aws

1st improvement

API should lie on another Folder

Model
app.py
requirement.txt

how to create requirement.txt:

pip freeze > requirements.txt

python -m venv myenv
myenv\Scripts\activate

pip install -r requirements.txt

2nd

@app.get('/') → human readable
home

@app.get('/health') → machine Readable
by aws

#MLFlow

Important improving API

1. Create a New Folder ✓
2. Field validator For city Feature ✓ → city titlecase
3. Add routes:
   a. Home ✓
   2) Heath check
4. Add Model version. ✓ → P
5. Seperation of logic. → Pydantic Model ✓ → clean seperation of concerns
   City tier
   ML logic
6. ~~Try catch~~
7. Add Confidence score; → rich → pydantic seperate ML logic config → confidence score high/medium/low → randomforest
8. Response Model. ✓

V.V.IMP

Docker in depth

Analogy

Maggie Noodle: → Noodle + Masala ← Maggie taiste the same Bacause of Masala.

If they provide receipe maggie test different:

So they Provide Masala → Container

Docker is a platform designed to help developers Build, share, & run Container applications

Why do we need Dockers:

Problem:

1.) Consistency Across Enviroments:

Applications often Behave differently in development, testing & production environments due to variation in configurations, dependencies and infrastructure.

Problem

Solution

Docker contains encapsulate all the necessary components ensuring the application runs consistently across all enviroments

Practical problem

Campusx → laptop price Predictor

gitclone → streamlit → rest

The code Break in a pipeline Because at the time of development different version of scikit-learn is used.

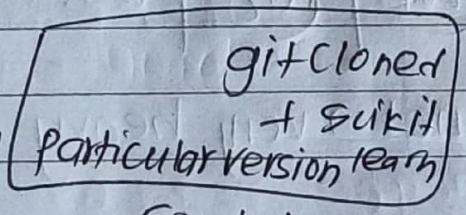
Fix1:

pip install scikit-learn -- version same as above

But this is not scalable & time consuming

Instead use dockor

Fix2



works fine ✓

Scalable ✓

efficient ✓

Problems

isolation

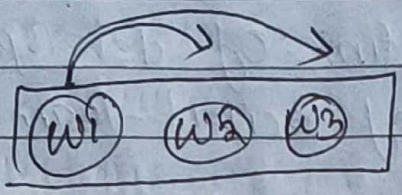
Problem: Running multiple applications on the same host can lead to conflict such as, dependency clashes or resources contention.

Solution Docker provides isolated enviroments For each application, preventing interference and ensuring stable performance.

Scanned with OKEN Scanner

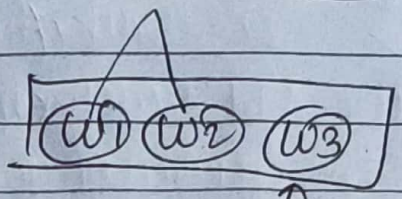
website → deploy

server → multiple



Hacking or Compromised W1

ideal wait

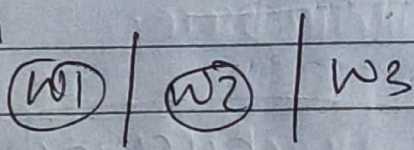


Heavy Traffic on W3

all hardware Resource ✓

so ~~docker~~

we need isolation



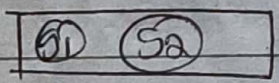
→ Virtual Machines → very heavy (own OS)

→ Docker

Problem 3 ★

Scalability

Morning low load



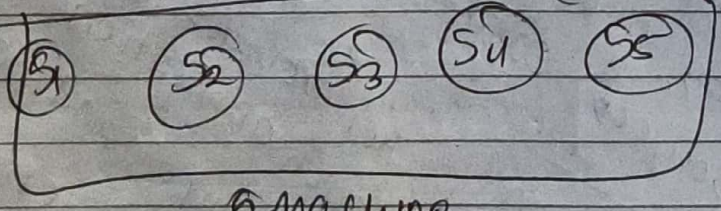
only 2 machine

~~Docker~~ containerize ✓

light weight copy

so easy to copy.

Evening high load



5 machine

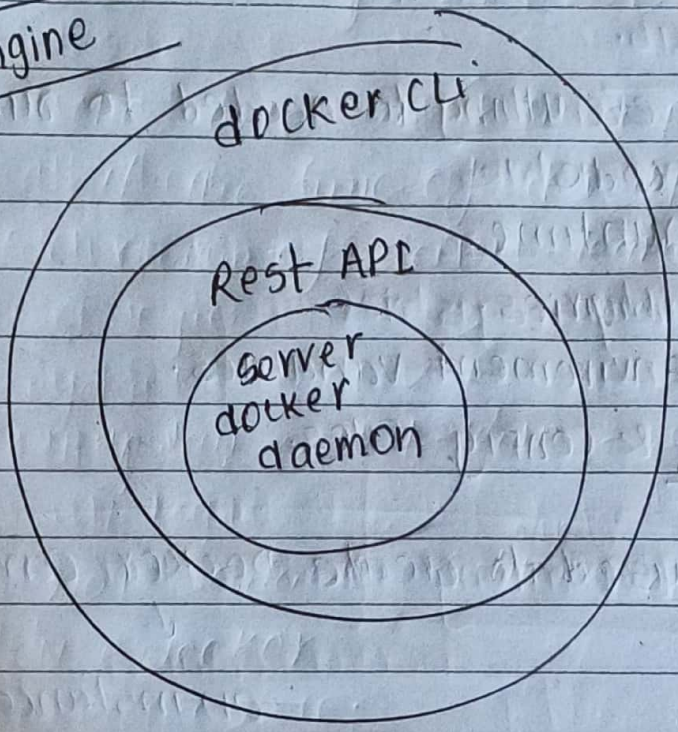
Scanned with OKEN Scanner

V. IMP

How Docker is used

Container engine

Docker engine



1) docker daemon (dockerd):

Func: the Docker daemon is the background service running on the host machine.
Manages Docker objects such as
↳ image, containers, N/w and volumes.

② Docker CLI

CLI is the tool that users interact with to communicate with the Docker daemon.

③ Rest API

DockerCLI ⟶ Docker deamon.

Scanned with OKEN Scanner

Docker image

Docker image → lightweight
→ stand-alone
→ executable software Package.

that includes everything needed to run a piece of software → code
→ runtime
→ libraries.
→ enviroment variables.
& Config Files.

images are used to create Docker containers.
↓
an instances of these images.

Components of Docker image

1. Base image → OS.
2. Application Code.
3. Dependencies.
4. Metadata.

Docker image lifecycle

1. Creation.
2. storage
3. Distribution.
4. Execution.

Scanned with OKEN Scanner

3rd

Dockerfile

text file that tells the docker daemon how to create the docker image

4th

Docker container

Docker image → execute → instance → container

Container is an instance of a docker image.

5th

Registry

Image → storage

A Docker Registry is a service that stores and distributes docker images.

Act like a repository where users can push, pull & manage Docker image.

Docker Hub is the most well-known public repo.

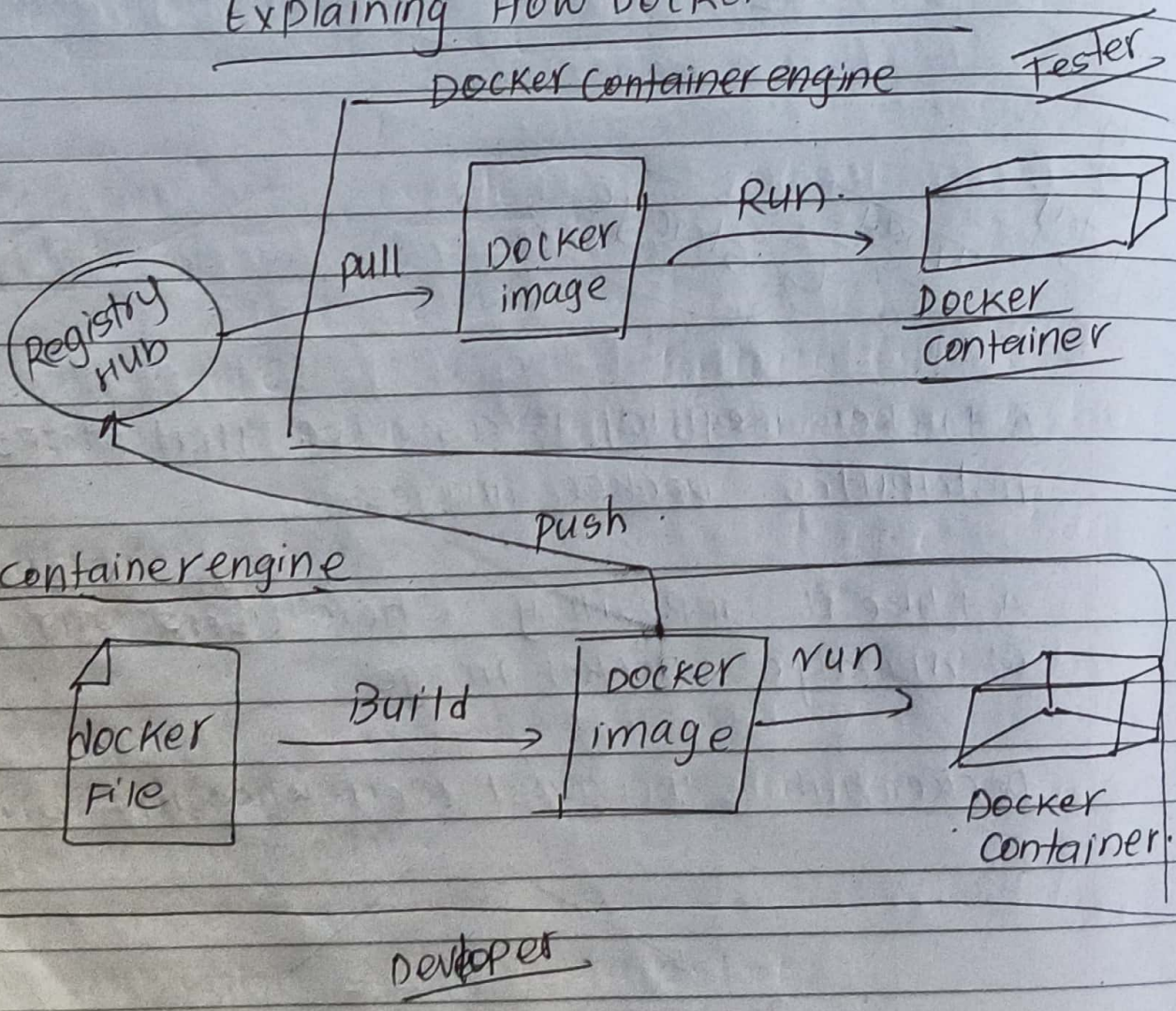
- Repositories (Docker image)
- Tags. (version of images).

Scanned with OKEN Scanner

Types of Docker registries:

1. Docker Hub.
2. Private registries.
3. third party registries

Explaining How Docker works:



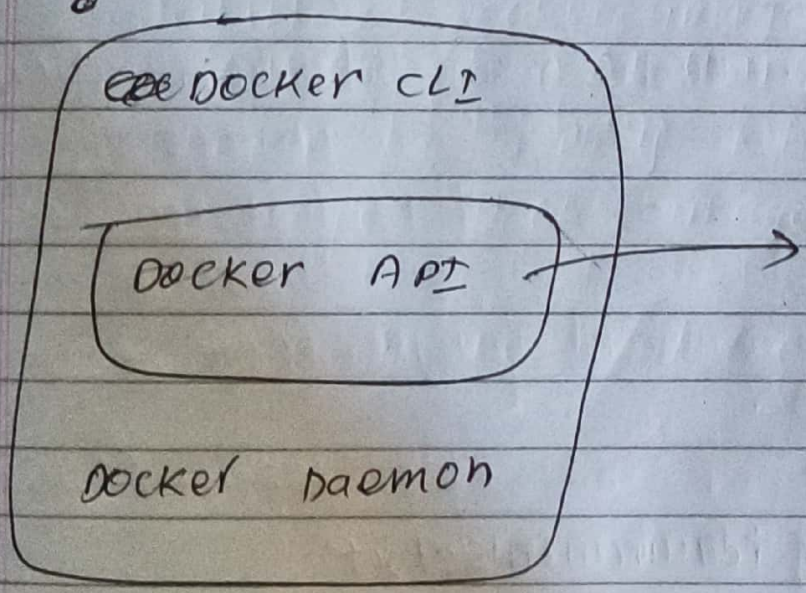
Use-cases

1. Microservices Architecture:
2. Continuous integration & continuous Development.
3. Cloud Migration
4. scalable WebApplication
5. Testing & QA.
6. Machine learning & AI
7. API Development & Deployment.

DOCKER DESKTOP

Docker engine

GUI



Concept of Ports

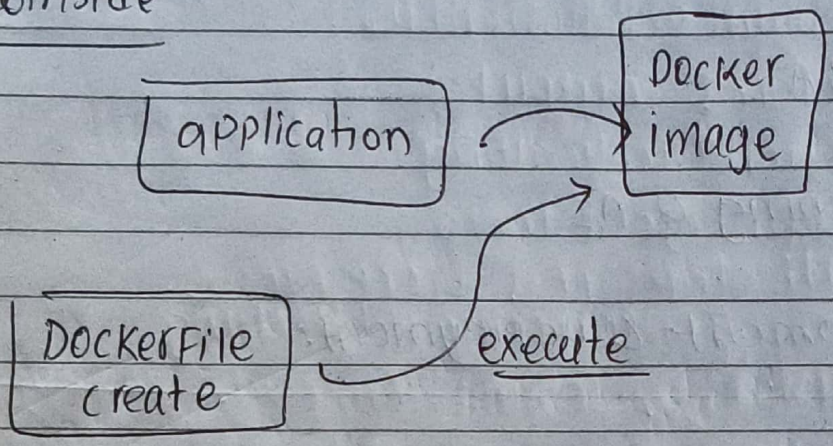
↑

docker run -p 8501: 8501 tweakster24/lappy

1st Coinside

Registry —pull→ Docker image —run→ Container.

2nd Coinside




```
#base image                    App with python.
  FROM python:3.9              #and also small os is installed

# Wordir
  WORKDIR /app

# COPY
  COPY . /app

# run
  RUN pip install -r requirements.txt
# Port
  EXPOSE 5000  # For Flask
```

Scanned with OKEN Scanner

Command.

CMD ["Python", "./app.py"]

Imp

```
IF __name__ == '__main__':
    app.run (host = '0.0.0.0', debug= True)
```

step2

docker build -t tweakster24/table .

username — Build context.

Docker image will start Building.

step3 execute image Container — used to map the port

docker run -P 8888:5000 tweakster24/table

8888 — my machine Port i.e outside the Docker.

5000 — port inside Docker image

Step4

Push the image into the dockerhub:

All in one workflow

check working fine

```
docker build -t rajujoshi/laptop .
                                  Build.

docker run -P 8501:8501 rajujoshi/laptop
```

Next step post in dockerhub

Authentication

1) ~~no~~ ~~docker~~ docker login

Authenticate credentials.

2) docker push rajujoshi/laptop

Now pull

```
docker pull rajujoshi/laptop

docker run -P 8501:8501 rajujoshi/laptop
```

Scanned with OKEN Scanner

2509 4124 0826

FastAPI
Docker

predict insurance → improved API

↓

Dockerize
(Docker image).

↓ push

DockerHUB

API Built ✓

↓

improvise → production grade ✓

↓

Dockerize ✓

↓

AWS

↓

Account

Done
FASTAPI